AD-065 · Quyền sở hữu dữ liệu và rollback

Đồng Vũ Hoàng Long · 23001901 · SE2026-T17
GAME-3D-03 – Small Arena · Thành viên nhóm · 3TC · Ngày khảo sát: 25/10/2026

> Tôi chịu trách nhiệm về toàn bộ nội dung bài làm và đã khai báo việc sử dụng AI.

1. Phạm vi và căn cứ thực tế

Đây là lát cắt máy chủ có thẩm quyền do Codex tạo mới cho bài cá nhân: xử lý lệnh đấm, lưu HP, chống lặp và ghi sự kiện trong một giao dịch. Không phải toàn bộ trò chơi. SRS v0.2 tại `Long_document/srs.md:29–37,358–371` xác nhận MVP ngoại tuyến, bot 10 HP, đấm thường trừ 1 HP; phần mạng là hướng tương lai (`:577–629`). Các mức 764 tác vụ, p95 ≤ 4,5 giây, lưu vận hành 7 ngày và phản hồi yêu cầu thay đổi 12 giờ đến từ đề AD-065, không phải NFR ngoại tuyến.

Kho gốc `tasdus-117/SE2026-T17`, nhánh mặc định `main@b5054a6` chỉ có README. Đã fetch và đọc hai nhánh tham khảo: `tasdus117@1257ad7` (Godot/C#/EF/PostgreSQL), `feature/system-skeleton@db886dd` (Go/UDP/pgx). Mã của đồng đội không được sao chép, merge hay cherry-pick. Worktree riêng bảo toàn `AD-065-prompt.md` và `Long_document/` chưa được theo dõi. Bằng chứng SHA đầy đủ: [E1].

SRS đề xuất Unity/C# cho client và không bắt buộc backend. Chọn Node 25.9.0 + SQLite 3.51.3 cho thí nghiệm độc lập: sẵn có, không cần thư viện runtime, có giao dịch thật và database dùng một lần. Docker có thể dùng nhưng không cần cho adapter này. Đây là quyết định phạm vi; chưa xác minh tích hợp Unity hay tính tương thích PostgreSQL.

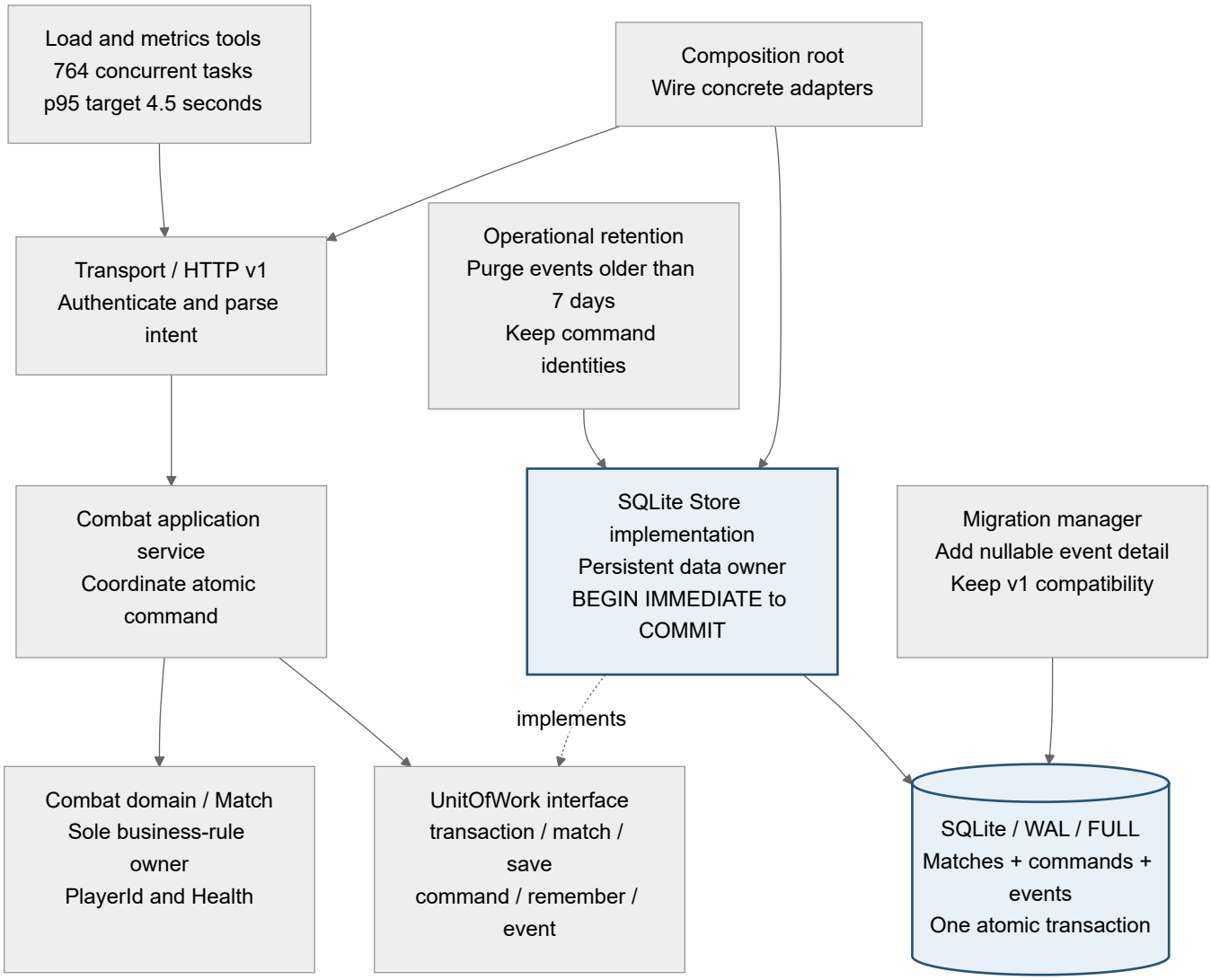
2. Đúng ba driver, theo thứ tự ưu tiên

Ưu tiên	Driver và nguồn	Hệ quả kiến trúc và lý do
D1	Độ trễ đồng thời: 764 tác vụ, p95 ≤ 4,5 s [đề AD-065]	API nhỏ, tải tách tiến trình, ghi cùng database; ưu tiên vì là thuộc tính chất lượng chính. Không suy ra năng lực sản xuất.
D2	Tin cậy khi đổi schema và rollback [tình huống AD-065]	Thêm cột nullable; xác minh sau triển khai; phục hồi app cũ trên schema mở rộng. Tránh DOWN mất dữ liệu.
D3	Nhất quán và quyền sở hữu [SRS FR-010/015/017; AD-065]	Match sở hữu luật; Store sở hữu dữ liệu; HP, định danh lệnh, sự kiện commit nguyên tử. Dữ liệu vận hành giữ 7 ngày; phản hồi thay đổi trong 12 giờ.

3. Một kịch bản chất lượng Performance

Nguồn: 764 tác vụ client thử nghiệm. **Kích thích:** gửi `POST /v1/combat`, 50% lệnh mới và 50% retry. **Môi trường:** máy Windows cục bộ, app/database ổn định sau 764 yêu cầu khởi động. **Đối tượng:** HTTP → dịch vụ → SQLite file. **Phản ứng:** xác thực, áp dụng đúng một lần hoặc trả kết quả đã lưu. **Đáp ứng đo được:** p95 từ lúc gửi đến đọc xong JSON ≤ 4,5 giây trong cửa sổ 10 vòng/tác vụ, 7.640 mẫu, tính cả yêu cầu lỗi; yêu cầu tỷ lệ lỗi 0. Cách tính và kết quả thực tại trang 6. Đây là kịch bản duy nhất của báo cáo.

2 · Module View và chiều phụ thuộc



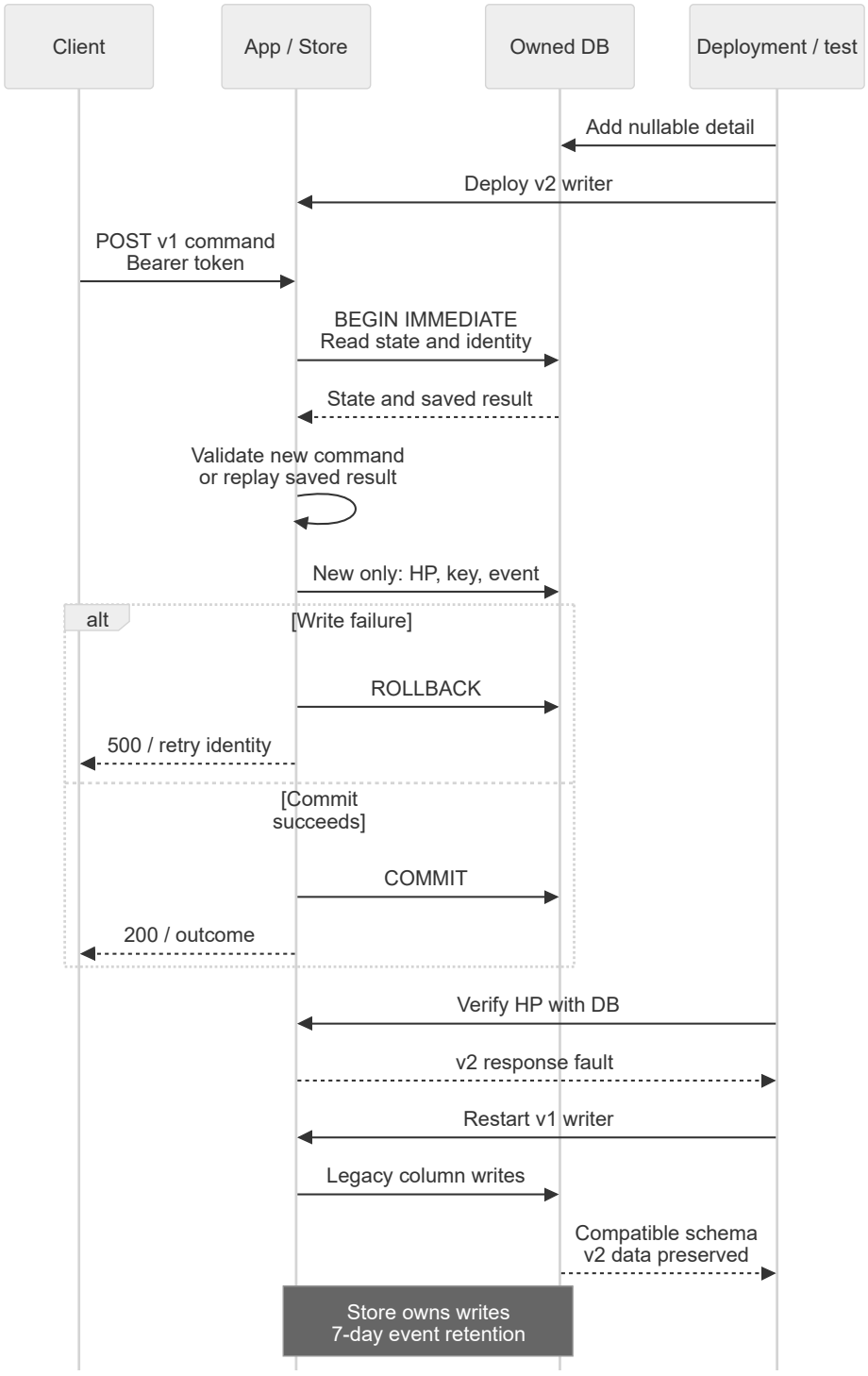
Hình 1. Mermaid Module View; ô xanh thể hiện quyền sở hữu lưu trữ và ranh giới giao dịch.

Transport xác thực token, giới hạn body và ánh xạ ý định; không tính sát thương. `CombatService` điều phối qua giao diện cấu trúc UnitOfWork. `Match`, `PlayerId`, `Health` tập trung luật và che giấu giá trị bên trong. Store hiện thực giao diện, là chủ duy nhất của các bảng `matches`, `commands`, `events`; migration chỉ thay cấu trúc, retention chỉ gọi xóa sự kiện đã hết hạn.

Đồ thị phụ thuộc hướng từ transport vào ứng dụng, từ ứng dụng vào domain và cổng giao dịch. Domain không import HTTP/SQLite. Composition root ghép các adapter cụ thể; đường implements nét đứt phân biệt cổng với hiện thực. Có tám trách nhiệm được chỉ rõ, trong đó công cụ tải đứng ngoài luồng nghiệp vụ.

`application.mjs:11-28` mở đơn vị công việc; `store.mjs:16-31` thực hiện `BEGIN IMMEDIATE` → đọc trạng thái/định danh → ghi HP + kết quả lệnh + sự kiện → COMMIT. Sai một bước thì ROLLBACK. Không phát sự kiện ra ngoài trước commit. `onCommitted` là observer tùy chọn, có thể lặp và mất; không được dùng làm nguồn thưởng bắt buộc. Domain data và danh tính lệnh không chịu TTL 7 ngày. [E2, E3]

3 · Runtime View: triển khai, phát hiện lỗi, rollback



Hình 2. Mô phỏng cục bộ hai chế độ writer; App/Store được gộp để đọc được trên A4.

Bất biến: cùng `(matchId, commandId)` chỉ gây sát thương một lần; HP nguyên trong [0,10]. Match kiểm tra chủ sở hữu và khoảng cách do server lưu; SQL hỗ trợ bằng khóa chính, UNIQUE, CHECK và FK. Lệnh mới khi HP = 0 bị 409; retry hợp lệ vẫn trả kết quả gốc, có thể khác HP hiện tại. [E2]

Hợp đồng v1: bearer token ánh xạ một actor thử nghiệm; body chỉ có `matchId`, `commandId`. Kết quả chứa version, hai ID, health và status. Lỗi 400/401/403/404/409/413/500; không nhận damage hoặc actor từ client. V2 chỉ thêm `events.detail` nullable. Test phát hiện phản hồi HP sai sau commit, dừng v2, chạy lại v1 trên cùng file; HP đi 10→9→8→7, dữ liệu v2 vẫn còn. Rollback app không đảo ngược sát thương đã commit. [E2, E4]

4 · ADR-065-01: một chủ ghi, một giao dịch

Trạng thái: chấp nhận cho thí nghiệm cá nhân; chưa là quyết định triển khai của nhóm. **Bối cảnh:** kho main chưa có game, SRS phân biệt MVP ngoại tuyến và mạng tương lai; tình huống AD-065 yêu cầu đổi schema có rollback, đồng thời đo 764 tác vụ. Khả năng vận hành của bài cá nhân có giới hạn. [E1]

So sánh hai phương án có thể triển khai

Tiêu chí	A · Monolith mô-đun + một DB	B · Combat service + event worker
Performance	Không hop mạng nội bộ; SQLite sync có thể chặn event loop	Có thêm mạng/queue; có thể tách tải worker nhưng phải đo
Nhất quán	Một commit cho HP, identity, event	Cần outbox, dedup consumer; nhất quán cuối cho projection
Sửa đổi	Domain/cổng tách adapter; phát hành chung	Worker đổi riêng; contract và migration phải phối hợp
Triển khai/rollback	Một app, schema cộng thêm giữ được writer cũ	Nhiều version producer/consumer cùng tồn tại; kiểm tra tương thích khó hơn
Độ phức tạp/chi phí	Một tiến trình, một file; phục hồi file cần thiết kế thêm	Broker, retry, giám sát độ trễ queue và nhiều dịch vụ
Năng lực nhóm	Phù hợp một lát cắt cá nhân có kiểm chứng	Chỉ hợp lý khi có nhu cầu phân tách vận hành và người phụ trách

Quyết định: chọn A. Combat application service là bên duy nhất yêu cầu đổi state; Match sở hữu luật và Store sở hữu ghi dữ liệu. Không dùng giao dịch phân tán. Phần B là lựa chọn tương lai có thật về thiết kế, chưa được triển khai hay benchmark trong bài này.

Lý do: tính nguyên tử chống lặp và tính tương thích schema có thể kiểm tra trực tiếp trong một database; lợi ích phân phối chưa có số đo chứng minh. Node/SQLite giúp thực thi bài ngay, dù không thể thay thế đầy đủ Unity authority hoặc PostgreSQL của các nhánh khác. Cổng UnitOfWork cho phép đổi adapter nhưng cần suite tích hợp mới cho DB mới.

Hệ quả: tăng cohesion nhờ luật trong Match/Health; giảm coupling trực tiếp với SQL ở ứng dụng. Đổi lại, giao dịch SQLite đồng bộ giữ khóa ghi và chặn event loop. COMMIT với `synchronous=FULL` bảo vệ độ bền tốt hơn, nhưng tăng độ trễ đĩa. Bỏ ghi dedup để giảm p95 sẽ phá bất biến khi retry; chọn giữ nhất quán, đo độ trễ rồi mới tối ưu. [E2, E5]

Chính sách triển khai: expand bằng nullable detail; chỉ contract khi hết cửa sổ rollback được thống nhất và không còn writer cũ. Không mặc định cửa sổ này là 12 giờ. Con số 12 giờ chỉ nghĩa vụ phản hồi yêu cầu thay đổi; chưa có hệ thống ticket tự động đo nghĩa vụ đó.

Kích hoạt xem xét lại: hai lần chạy 5 phút đại diện liên tiếp ở 764 tác vụ có p95 >4,5 s hoặc lỗi >0,1%; event-loop lag p95 >50 ms trong 5 phút; yêu cầu nhiều writer độc lập trên cùng tập match; hoặc danh tính lệnh vượt 1 GB/database. Khi đó đo contention, cân nhắc PostgreSQL/shard theo match hay outbox trước khi tách service. Đây là ngưỡng đề xuất, không phải kết quả đã đo. [E6]

5 · Chất lượng mã và refactoring có dấu vết

Baseline thực, không gán nợ mới cho đồng đội

Commit `fd4df4c4f13734eb812063e1b1861d4523d4a68b` lưu feature đấm thuần và ba test đặc tả trước khi sửa. `src/combat.mjs:1–6` nhận `health`, `attacker`, `owner` dưới dạng number/string. Mã này được tạo trong phiên AI cho bài cá nhân. Không phải phát hiện smell có sẵn trong mã của nhóm. Commit `1ff103387097911312fdf0666bf993ad384a0e98` lưu hiện thực sau refactor và kết quả kiểm thử. Tiền tố đường dẫn `src/` và `test/` tại trang này là `assignment/AD-065/implementation/`.

Vấn đề và bằng chứng	Rủi ro / phạm vi ảnh hưởng	Bước xử lý và an toàn hồi quy
Q1 · Primitive obsession, baseline `combat.mjs:1–6`	HP và damage đều có thể là số; string chưa phân biệt danh tính. Luật dễ bị lặp ở caller; mất nghĩa miền.	Đã xử lý: `domain.mjs:5–36` dùng PlayerId, Health, Match; private field che giấu state. Giữ `strike` làm adapter hành vi. Ba test characterization giữ nguyên.
Q2 · Coupling lịch thực thi, `store.mjs:16–29`	SQL đồng bộ và HTTP dùng cùng event loop; thao tác đĩa chậm ảnh hưởng mọi request trong process, dù khác match.	Chưa xử lý: đo event-loop lag và khóa trước; chuyển adapter sang worker hoặc PostgreSQL async khi có căn cứ. Giữ suite atomicity + 764 retry + benchmark khi thay adapter.
Q3 · Giao diện observer, `transport.mjs:22–23`	Callback chạy sau mọi response thành công, kể cả replay; mất hoặc lặp thông báo. Ghép phần thưởng vào callback sẽ sai.	Chưa xử lý: contract ghi rõ best effort; nếu cần side effect bắt buộc, dùng outbox/consumer dedup. Test hiện tại chứng minh observer lỗi không hủy commit; cần test crash/replay consumer mới.
Q4 · Vòng đời dữ liệu, `migrations.mjs:8–10`; `retention.mjs:3–4`	Commands không hết hạn: đúng chống lặp nhưng tăng dung lượng, ảnh hưởng index và vận hành.	Chưa xử lý: đóng/archival match rồi từ chối lệnh muộn trước khi xóa danh tính. Test purge hiện có bảo đảm retry cũ không gây damage mới.

Cohesion: arithmetic và điều kiện HP tập trung trong Health, quyền đánh trong Match; transport chỉ biết contract. **Information hiding:** private state không có setter công khai; ứng dụng chỉ nhận hành vi và kết quả. **Dependency direction:** domain không biết Store; UnitOfWork được truyền vào thay vì dịch vụ tự mở DB. Hợp đồng cấu trúc JavaScript chưa được kiểm tra kiểu tĩnh, là giới hạn thực tế chứ không được gọi là “đã chứng minh type safety”.

Trước/sau: cùng mọi HP sống 1..10 trả HP−1; sai owner vẫn 403 trước kiểm tra kết thúc; HP=0 vẫn 409; health hỏng vẫn lỗi đóng. Ba test ban đầu qua cả hai commit. Phạm vi chứng minh là các trường hợp đã đặc tả với ID hợp lệ, không phải mọi input tùy ý của helper cũ. API v1 giữ shape, version và error semantics trong suite tích hợp. [E2, E3]

6 · Xác minh và sổ nợ kỹ thuật

Kết quả đã thực thi

`node --test test/*.test.mjs`: **10 pass, 0 fail, 0 skip**, gồm 3 characterization và 7 integration. Kiểm tra HP/owner/range/terminal, CHECK/FK, intent-only API, 764 retry cùng ID, fault giữa ghi HP và event, đóng/mở DB, migration v1→v2→writer v1, cutoff 7 ngày và replay sau purge. `node tools/check.mjs`: cú pháp và whitespace 13 module pass; Node không cần bước biên dịch riêng. Không chạy Unity, PostgreSQL, WAN, kiểm tra kiểu tĩnh hay soak dài. [E3]

Hai lỗi đầu được giữ trong `evidence/tests/initial-failures.md`: burst 764 kết nối cùng event loop gây ECONNREFUSED; tăng backlog riêng chưa giải quyết. Chuyển fixture server sang process riêng giữ nguyên 764 tác vụ và mọi assertion thì pass. Lỗi EPERM cleanup do DB v1 còn mở được sửa bằng đóng handle trước xóa temp. Không bỏ assertion hoặc đổi tải để che lỗi.

Đo tải thực, có giới hạn

Ryzen 7 5800HS, 16 CPU logic, RAM 16.541.487.104 byte; Windows `10.0.26200`; Node `v25.9.0`; SQLite `3.51.3`, file WAL/FULL. Client và server là hai process trên cùng máy, HTTP loopback; cố định một owner và khoảng cách hợp lệ. **764 tác vụ closed-loop**, mỗi tác vụ một request đang chờ, không think time; 50% lệnh mới/50% retry; 5 lệnh mới/match. Warm-up 764 mẫu trong **876 ms**, 0 lỗi. [E5]

Đo lúc `2026-10-25T16:25:49.859Z` trong **4,299 s**, **7.640 mẫu**, **0 lỗi (0%)**, **1.776,95 request/s**; p50 **420,75 ms**, p95 **583,74 ms**, p99 **658,32 ms**. Sắp xếp độ trễ tăng dần, nearest-rank `sorted[ceil(p×N)−1]`; thời gian từ gửi đến đọc xong JSON, bao gồm mẫu lỗi nếu có. **Đạt mục tiêu p95 ≤4.500 ms trong thí nghiệm này**. Không chứng minh p95 trong sản xuất, tải mở, nhiều writer hoặc đủ dài để thấy saturation. Raw samples và command tái lập tại [E5].

Nợ tồn tại và thứ tự xử lý

Điểm = tác động × cấp bách / công sức; mỗi thang 1–5. Đây là ước lượng AI có giải trình, không phải chấm điểm của sinh viên hay số đo chi phí. Có thể đổi thứ tự khi có số liệu mới.

Nợ	Bằng chứng / tác động	I/U/E → điểm	Hành động / hồi quy
TD1	Q2: writer đồng bộ chặn các request	4/4/3 → 5,33	Đo lag/lock; adapter async khi cần; chạy atomicity và tải.
TD2	Q3: observer có thể mất/lặp	4/3/3 → 4,00	Outbox khi có effect bắt buộc; test crash/replay.
TD3	Q4: commands tăng không giới hạn	3/3/3 → 3,00	Archival với từ chối lệnh muộn; test replay sau purge.

TD1 đứng đầu vì đường ghi nằm trên mọi command, trực tiếp đe dọa D1; TD2 chưa gây sai kết quả miền vì observer không bắt buộc; TD3 cần dữ liệu dung lượng dài hạn. Q1 đã giải quyết, không ghi thành nợ còn mở. [E6]

7 · Review AI, traceability và kết luận

Review đơn phiên, có phê bình

Prompt thật yêu cầu kiểm tra boundary, rollback, refactoring, retention và tải tại commit `1ff1033`; lưu tại `evidence/ai/design-review-prompt.md`. Output do chính Codex phiên này tạo, không có reviewer độc lập. Đề xuất 1: áp dụng TTL 7 ngày cho cả commands và events. **Bác bỏ phần commands** vì xóa identity làm retry cũ gây damage mới; test retention xác minh identity còn sau purge (`test/integration.test.mjs:135-148`). Đề xuất 2: p95 tốt đủ để nhân nhiều writer. **Bác bỏ suy luận**, sửa thành đo contention trước: Store dùng BEGIN IMMEDIATE sync; tải chỉ có một writer và loopback. [E6]

Giữ đề xuất một transaction và schema cộng thêm vì test fault/restart/rollback hỗ trợ. Chưa chắc chắn về physics/cooldown, WAN, rotation token, outbox, disk contention và PostgreSQL. Không có xác nhận kiểm tra trực tiếp của sinh viên trong phiên; toàn bộ mã và phân tích mới do AI hỗ trợ và được khai báo. Phụ lục chứa prompt gốc đầy đủ, prompt review và output thực.

Ma trận truy vết

Tiền tố **I** = `assignment/AD-065/implementation/`; **E** = `assignment/AD-065/evidence/`. Mã mới/refactor ở commit `1ff103387097911312fdf0666bf993ad384a0e98`; baseline smell `fd4df4c4f13734eb812063e1b1861d4523d4a68b`.

Yêu cầu → quyết định	Boundary/contract → mã chính xác	Test → kết quả / chứng cứ
D1: 764, p95≤4,5 s → một writer, đo thật	HTTP v1 → I`tools/load.mjs:16-58`; I`src/config.mjs:1`	7.640 mẫu, p95=583,74 ms; E`performance/load.json`
D2: rollback schema → expand/giữ cột	Writer v1/v2 → I`src/migrations.mjs:1-20`; I`src/store.mjs:23-25`	I`test/integration.test.mjs:111-133` pass; E`rollback/result.md`
D3: chống lặp/HP → giao dịch nguyên tử	Match + UnitOfWork → I`src/application.mjs:11-28`; I`src/domain.mjs:17-35`	I`test/integration.test.mjs:40-80,95-109` pass; E`tests/final-tests.txt`
Vận hành: 7 ngày → purge event, giữ identity	Retention → I`src/retention.mjs:3-4`	I`test/integration.test.mjs:135-148` pass
Phản hồi thay đổi: 12 giờ → quy tắc vận hành	I`src/config.mjs:1`; I`CONTRACT.md`	Test assert cấu hình pass; thực thi quy trình ticket chưa xác minh
Primitive obsession → Health/PlayerId	I`src/combat.mjs:1-6`; I`src/domain.mjs:5-26`	Ba characterization không đổi, pass trước/sau; E`tests/`

Kết luận trong phạm vi bằng chứng

Lát cắt chạy được chứng minh tính nguyên tử và khả năng phục hồi app cũ với schema cộng thêm. Không đánh đồng bản mẫu này với máy chủ 3D hoàn chỉnh. Các nhu cầu chưa thực thi phải qua đo và review trước khi mở rộng; sinh viên cần tự đọc, hiểu và chịu trách nhiệm về nội dung theo tuyên bố trang đầu.

Tài liệu truy chứng: [E1] `evidence/repository-baseline.md`, `evidence/source-analysis.md` (SHA và nguồn SRS); [E2] `implementation/CONTRACT.md`, `src/` tại commit nêu trên; [E3] `evidence/tests/final-tests.txt`, `static.txt`, `initial-failures.md`; [E4] `evidence/rollback/result.md`; [E5] `evidence/performance/load.json`, `implementation/tools/load.mjs`; [E6] `evidence/ai/design-review-output.md`, ADR/sổ nợ trên trang 4–6. Tất cả dưới `assignment/AD-065/` trừ SRS địa phương đã ghi rõ. Không sử dụng trích dẫn học thuật bên ngoài chưa kiểm tra.

Phụ lục A · AI Usage Log

Thông tin và mức độ sử dụng AI

Sinh viên: Đồng Vũ Hoàng Long, 23001901; SE2026-T17; GAME-3D-03 – Small Arena; AD-065, 3TC, thành viên nhóm. Ngày phiên làm việc: 25/10/2026 (Asia/Bangkok). Công cụ: Codex CLI; không tự khẳng định phiên bản model nội bộ. Không có hội thoại reviewer độc lập hoặc xác nhận cá nhân của sinh viên.

Người dùng yêu cầu nguyên văn: “Read AD-065-prompt.md completely and execute all instructions.” Chỉ dẫn dài được đọc từ AD-065-prompt.md và được lưu đầy đủ trong original-user-prompt.md, đồng thời in ở Phụ lục C. Bản sao này là yêu cầu thật được cung cấp, không phải prompt do AI tái dựng. Các file báo cáo cũ tại Long_document không được dùng làm bài phân tích mới.

Công việc AI đã thực hiện

AI đọc nhánh, SRS và môi trường; chọn Node/SQLite; viết mã mới, tests và load harness; refactor primitive baseline; tạo Mermaid; phân tích ADR, nợ, truy vết; viết tiếng Việt và dựng PDF. Tất cả mã/phân tích mới đều có hỗ trợ AI, không được trình bày như sinh viên tự viết không dùng AI. Mã baseline độc lập ở fd4df4c; mã sau refactor ở 1ff1033; SHA đầy đủ trong trang 5 và repository-baseline.md.

Các lệnh thực thi gồm git fetch/worktree/commit, node --test, node tools/check.mjs, node tools/load.mjs, Mermaid CLI, Chrome PDF và Poppler. Kết quả thật được lưu tại evidence/tests, evidence/performance, evidence/rollback và evidence/pdf. Không tạo số đo hay xác nhận kiểm tra của con người. Test không chạy được hoặc ngoài phạm vi được đánh dấu rõ trong báo cáo và verification-summary.md.

Review có thật trong cùng phiên

Prompt phụ được lưu nguyên văn ở design-review-prompt.md; output thực do chính Codex phiên này tạo tại design-review-output.md, in trong Phụ lục B. Hai đề xuất được nêu rồi đánh giá: xóa command identity theo TTL vận hành bị bác bỏ vì phá chống lặp; suy ra scale nhiều writer từ p95 cục bộ bị bác bỏ vì chưa đo contention. Giữ giao dịch đơn và schema cộng thêm dựa trên test. Đây là tự review có AI hỗ trợ, không giả lập nhiều reviewer.

Sai sót, sửa chữa và giới hạn

Ban đầu concurrency fixture thất bại ECONNREFUSED; thêm backlog chưa đủ. Tách server khỏi process tải đã giải quyết fixture mà giữ nguyên 764 tác vụ. Cleanup rollback gặp EPERM do handle DB mở; đã sửa thứ tự đóng. Mermaid sequence gặp lỗi cú pháp do dấu chấm phẩy trong label; đã thay văn bản và render lại. Lỗi và cách sửa được lưu, không bỏ qua test để tạo kết quả đẹp.

Chưa kiểm chứng Unity, PostgreSQL, WAN, nhiều writer, soak dài, ticket SLA 12 giờ, phục hồi production hoặc quyết định của nhóm. Chỉ số nợ và review trigger là đề xuất AI, không phải nhận xét cá nhân được gán cho sinh viên. Trách nhiệm cuối cùng thuộc người nộp; phiên này không có bằng chứng sinh viên đã tự kiểm tra mọi lập luận.

Phụ lục B · Prompt và output review thực

```
# Actual focused prompt, single Codex session

Review the original AD-065 implementation at commit 1ff103387097911312fdf0666bf993ad384a0e98.
Assess data ownership, transaction atomicity, primitive refactoring, schema rollback,
retention and concurrent-load evidence. Consider whether (1) retention should apply
to command deduplication as well as operational events, and (2) the passing SQLite
load result justifies deploying multiple writer replicas. Give proposals, then
evaluate them critically against exact code and executed tests. Do not invent an
independent reviewer or personal student approval. Preserve uncertainties.
# Actual Codex-assisted review output
This text is the actual review generated in this session in response to design-review-prompt.md;
it is not a transcript of a separate model or a human student's judgment.
Proposal 1: unify the operational retention policy by expiring both event and command rows after 7 days.
Decision: REJECT the command-row part. commands is durable domain correctness data:
application.mjs:15-19 returns stored responses before applying damage; migrations.mjs:9-11
defines the match-scoped primary key. Removing identity records permits old requests
to apply damage twice. Keep the event-only deletion in retention.mjs:3-4; the exact
cutoff test integration.test.mjs:135-148 verifies retries after purge. A future bounded
dedup policy needs an explicit match archival/rejection rule, not a silent TTL.
Proposal 2: treat the measured p95 below 4.5 seconds as sufficient evidence to add multiple writer replicas.
Decision: REJECT that inference, MODIFY the scaling recommendation to measure contention
first. store.mjs:16-17 uses synchronous BEGIN IMMEDIATE, and load.json explicitly records
one server process and a loopback client. 764 tasks and 7640 samples do not test multiple
writers, WAN latency or sustained arrivals. Partition matches into independently owned
shards only if measurements require it; do not share one SQLite writer file across replicas.
Accepted observation: transaction contains HP, command response and durable event; restart
and injected-write-failure tests establish the intended local atomicity. No distributed
transaction is justified by this slice. Optional observer is best effort and fires again on
replay; it must not drive mandatory rewards or external exactly-once effects.
Refactoring review: baseline fd4df4c contains primitive strike arguments; Health and PlayerId
now centralize valid range/equality. Existing characterization tests are unchanged. This
supports behavior preservation for the captured valid-ID and stored-health cases only,
not an exhaustive proof across arbitrary undocumented baseline inputs.
Remaining uncertainty: SQLite writer contention on multiple processes, disk saturation,
Unity client integration, movement/occlusion/cooldown validation, PostgreSQL compatibility,
long-running event-loop delays, secret rotation and observer delivery are not verified.
The student has not personally reviewed or approved this AI-produced analysis in this session.
```

Phụ lục C · Nguyên văn chỉ dẫn được cung cấp

You are Codex CLI acting as a senior software architect, backend engineer, test engineer, and technical report author.
Your task is to complete my individual Software Architecture assignment, AD-065, using the REAL GitHub repository, while producing an original, independently implemented code slice and a thoroughly verified Vietnamese PDF report.
DO NOT stop at analysis, planning, or documentation. Execute the full workflow: inspect repository → create my branch → implement original code → run tests → collect evidence → conduct AI design review → create diagrams → generate PDF → validate deliverables.
Work autonomously, within the available environment and permissions. Inspect command failures and repair them where possible. Do not claim successful execution when it has not occurred.
==========
1. STUDENT AND ASSIGNMENT IDENTITY
==========
Student: Đồng Vũ Hoàng Long
Student ID: 23001901
Team: SE2026-T17
Project: GAME-3D-03 – Small Arena
Variant: AD-065
Role: Team member
Credits: 3TC
Repository:
https://github.com/tasdus-117/SE2026-T17
Required final PDF filename:
BT2_23001901_AD-065.pdf
Required final location:
assignment/AD-065/submission/BT2_23001901_AD-065.pdf
The final report must be written in Vietnamese.
EXCEPTION:
Both Mermaid diagrams must contain ONLY English text, including node names, labels, legends, edge labels, titles, and annotations.
The report must contain at most 7 A4 pages of MAIN CONTENT.
The AI Usage Log appendix may extend beyond those 7 pages.
The first page MUST include this exact Vietnamese declaration:
"Tôi chịu trách nhiệm về toàn bộ nội dung bài làm và đã khai báo việc sử dụng AI."
Academic integrity is mandatory.
This is an individual assignment. Never copy another student's diagrams, ADR, debt register, analysis, or source code.
AI-generated work must be clearly disclosed. Do not manufacture personal judgments, experiments, source evidence, or student verification.
==========
2. EXACT AD-065 VARIANT
==========
Analyze the authoritative multiplayer 3D server of GAME-3D-03.
Architecture incident:
A new deployment changes the database schema, but the system must support rollback when post-deployment verification fails.
Primary boundary:
Data ownership and transaction boundary.
Mandatory code smell:
Primitive obsession that destroys domain meaning.
Primary quality attribute:
Performance, specifically response time under concurrent load.
Exact parameters:
- Concurrent users/tasks: 764
- p95 target: <= 4.5 seconds
- Operational data retention: 7 days
- Change request response window: 12 hours
These values MUST appear correctly throughout:
- architecture drivers
- quality attribute scenario
- diagrams where appropriate
- contracts

Phụ lục C · Nguyên văn chỉ dẫn được cung cấp

- implementation/configuration
- tests
- performance evidence
- ADR
- traceability matrix
- final PDF
Never replace 764 with 786, 568, or another value.
Do not claim the 4.5-second target has been achieved unless a real load test provides supporting measurements.
State the exact load-test environment, request mix, duration, sample size, and p95 calculation method when reporting performance.
The 12-hour value describes the change-request response obligation, not necessarily implementation completion time.
==========
3. REPOSITORY INVESTIGATION
==========
Before modifying anything:
1. Inspect the current Git working directory.
2. Confirm the origin remote.
3. Inspect all local and remote branches.
4. Record git status and existing uncommitted changes.
5. Identify the default branch and its commit SHA.
6. Fetch the following branches read-only:
   origin/tasdus117
   origin/feature/system-skeleton
7. Inspect their file trees, relevant source, tests, and documentation.
8. Search the repository for:
   srs.md
   Long_document
   README.md
   architecture documents
   technology descriptions
   requirements
   game/server code
   persistence code
   deployment configuration
   test infrastructure
9. Locate and read the actual SRS, including the document under Long_document if it exists.
If the SRS is missing, search repository history and available local files. Do not fabricate its contents.
Use the SRS technology choices as a reference, but do NOT automatically implement the complete offline MVP described there.
Distinguish:
A. Confirmed project requirements
B. Confirmed existing implementation
C. Proposed AD-065 architecture
D. New code created for this assignment
E. Verified test results
F. Unverified hypotheses
Record repository inspection evidence:
- Branch names
- Exact commit SHAs
- Examination date
- Source paths
- Relevant line ranges
- Technology stack
- Observed implementation limitations
Important:
The two branches below contain teammate code and are READ-ONLY REFERENCES.
https://github.com/tasdus-117/SE2026-T17/tree/tasdus117
https://github.com/tasdus-117/SE2026-T17/tree/feature/system-skeleton
NEVER:

- Modify these branches
- Cherry-pick their implementation commits
- Copy their source files
- Copy their design diagrams
- Copy their ADRs or debt registers
- Present their source code as my work
- Merge their branches into my individual branch

You may inspect them to understand repository context and identify architectural constraints.
==========
4. CREATE MY OWN BRANCH
==========
Create this new branch:
assignment/AD-065-23001901
Prefer creating it from the official default branch at a recorded base commit, NOT from either teammate implementation branch.
Verify the resulting branch does not inherit copied implementation code from those branches.
Preserve all existing user changes.
If the workspace is dirty, use a safe Git worktree or another isolated method rather than deleting or overwriting changes.
If the branch already exists:
- Inspect it
- Preserve existing user changes
- Continue safely
- Never force-reset without explicit authorization

All new implementation and assignment artifacts belong to my own branch.
Use descriptive commits, for example:
- chore(ad065): establish independent assignment baseline
- feat(ad065): implement authoritative server slice
- test(ad065): verify invariants and schema rollback
- docs(ad065): add architecture evidence and report

Push only my new branch if authentication and permissions permit.
Do not open or merge a pull request unless needed for the workflow and explicitly authorized.
==========
5. IMPLEMENTATION SCOPE AND TECHNOLOGY
==========
I do NOT want to rebuild an entire multiplayer 3D game solely for this assignment.
I want an original, small but credible, executable vertical slice of the authoritative server.
It must directly support the assignment's architecture analysis.
Select the smallest cohesive feature that provides strong evidence for:
- Server-authoritative validation
- Domain invariants
- Explicit data ownership
- Database transactions
- Schema evolution
- Safe rollback
- Performance under concurrent load
- Primitive obsession
- Refactoring safety

Suggested feature:
Authoritative combat command processing with a persistent match/combat outcome and operational event logging.
This is a suggestion, not a mandatory implementation.
First inspect the real SRS and technology constraints.
Prefer its documented technology stack when suitable.
Examples:
- Unity/C# client integration
- Go authoritative server
- PostgreSQL persistence

But do NOT force these technologies if actual documented requirements contradict them.
Explain the technology selection with evidence.

Phụ lục C · Nguyên văn chỉ dẫn được cung cấp

Prefer a modular monolith for this individual vertical slice IF justified by coupling, transaction consistency, deployment cost, and testability.
Compare it with at least one realistic alternative.
Do not choose microservices merely because they are supposedly easier to scale.
Implement at least five meaningful modules/components, for example:
1. Transport/API
2. Command application service
3. Combat domain
4. Match aggregate/state management
5. Persistence adapter
6. Schema migration manager
7. Operational event/retention module
8. Metrics and load-test support
Select the actual modules based on implementation needs.
Ensure their responsibilities and dependency directions are explicit.
Domain rules must not be hidden in HTTP handlers, UI code, or arbitrary database helper functions.
==========
6. DATA OWNERSHIP AND TRANSACTION BOUNDARY
==========
Define the authoritative owner of match/combat state.
Document:
- Which module may mutate domain state
- Which module owns each persistent data set
- Where a transaction starts and commits
- Which operations must be atomic
- Which events occur after commit
- How other modules access the data
- How concurrent commands are handled
Provide a specific invariant.
For example:
A combat command must not apply damage twice for the same command identity.
Or:
A player's health must remain within its valid domain range.
Choose an invariant appropriate to the actual implementation.
Enforce the invariant at server/domain level, supported by database constraints or transactional mechanisms where applicable.
Do not depend only on frontend validation.
Implement a versioned API or event contract with documented:
- Inputs
- Outputs
- Command identifiers
- Domain identifiers
- Error semantics
- Versioning
- Compatibility expectations
Avoid distributed transactions unless there is compelling evidence that they are needed.
==========
7. SCHEMA EVOLUTION AND ROLLBACK
==========
Implement a realistic schema migration demonstration.
Use an expand-and-contract strategy when justified.
Demonstrate:
1. Starting schema
2. Additive migration
3. Application compatibility
4. Deployment of the new application version
5. Post-deployment verification
6. Failure detection
7. Application rollback

Phụ lục C · Nguyên văn chỉ dẫn được cung cấp

8. Database compatibility after rollback
Do not assume that automatically reverting a database migration is always safe.
If backward data migration is unsafe, explicitly document it and retain an application-compatible schema until the rollback window closes.
Include a practical failure scenario.
Provide automated tests where feasible for:
- Migration execution
- Old/new schema compatibility
- Rollback to previous application behavior
- Transaction atomicity
- Domain invariants
- Duplicate command handling
- Failure recovery
For any failure simulation, isolate test data from real systems.
NEVER perform destructive operations against an unknown production database.
Use local disposable test databases or containers.
==========
8. PRIMITIVE OBSESSION AND REFACTORING
==========
This assignment requires real code evidence, not hypothetical code smells.
Design and implement a small initial working feature.
Capture its baseline BEFORE refactoring.
For a genuinely relevant portion, permit a limited primitive-based design that represents a meaningful refactoring opportunity.
Do not deliberately introduce unsafe vulnerabilities or broad technical debt.
Possible domain types:
PlayerId
MatchId
CommandId
Health
Damage
MatchStatus
Choose appropriate types from the actual domain.
Before refactoring:
- Preserve the exact original code at a traceable baseline commit
- Record the file paths and line ranges
- Write characterization/regression tests
- Establish observable behavior
Then refactor primitive values into meaningful domain types or value objects.
After refactoring:
- Preserve externally observable behavior
- Preserve API compatibility where required
- Preserve error semantics
- Run the regression tests again
- Document changes in cohesion/coupling/information hiding
Avoid turning an invented before-refactoring issue into a claim about pre-existing team code.
Clearly distinguish:
"Newly authored baseline code"
from
"Pre-existing repository technical debt."
The final report must analyze at least three ACTUAL, verified code-quality concerns.
For every issue provide:
Evidence
→ Risk / impact surface
→ Refactoring step
→ Regression safety
Analyze concerns involving:
- Cohesion
- Coupling

Phụ lục C · Nguyên văn chỉ dẫn được cung cấp

- Information hiding
- Dependency direction
- Technical debt
At least one concern MUST be primitive obsession.
Prefer genuine trade-offs or measurable risks over artificially manufacturing defects.
If fewer than three justified problems can be established, explain the evidence gap and do not invent findings.
==========
9. ARCHITECTURE DRIVERS
==========
Identify EXACTLY three prioritized architecture drivers.
Select them from actual requirements, quality attributes, and project constraints.
The selected drivers must cover:
- Concurrent-load performance
- Deployment/schema rollback and reliability
- Data consistency/ownership or another strongly supported domain constraint
Explain:
- Source requirement
- Architectural consequence
- Priority
- Rationale
Create exactly one complete Performance quality attribute scenario with:
Source
Stimulus
Environment
Artifact
Response
Measurable response
The scenario must explicitly use:
764 concurrent users/tasks
p95 <= 4.5 seconds
Specify the measured request/operation and measurement window.
Explain a concrete trade-off between Performance and another quality attribute, such as Consistency, Reliability, or Modifiability.
Avoid generic explanations.
==========
10. MERMAID DIAGRAMS
==========
Create EXACTLY two primary diagrams.
Diagram 1:
Module View
Diagram 2:
Runtime View
Save the Mermaid source as:
assignment/AD-065/diagrams/module-view.mmd
assignment/AD-065/diagrams/runtime-view.mmd
ALL text inside both diagrams MUST be English.
Diagram 1 requirements:
- At least five modules/components
- Clear responsibilities
- Dependency directions
- Main business-rule owner
- Persistent data owner
- Transaction boundary
- Clear distinction between interfaces and implementations
Diagram 2 requirements:
- Requests
- Events
- Data flow
- Deployment with a schema change

- Post-deployment verification
- At least one failure path
- Application rollback
- Database compatibility
- Transaction and data ownership boundaries

Use Mermaid flowchart, sequenceDiagram, or another suitable Mermaid type.
Prefer:
- Flowchart for Module View
- Sequence Diagram for Runtime View

The diagrams MUST represent the actual proposed/implemented architecture, not fictional infrastructure presented as existing.
Use clear labels and avoid dense crossing arrows.
Render Mermaid into high-quality SVG files.
Validate the Mermaid syntax automatically.
Embed the rendered diagrams in the PDF.
Both diagrams must remain legible on printed A4 paper.
Inspect their rendered output to detect clipping, tiny text, overlapping labels, or incorrect arrows.
Do not merely paste Mermaid source code into the PDF.
==========
11. ADR AND ARCHITECTURAL TRADE-OFFS
==========
Compare at least two reasonable architecture approaches.
Examples:
A. Modular monolith with one transactional database
B. Service-oriented or event-driven design
You may select alternatives appropriate to the actual system.
Compare them in terms of:
- Performance
- Consistency
- Modifiability
- Deployability
- Schema rollback
- Complexity
- Operational overhead
- Team capacity

Create one complete ADR with:
- ADR ID and title
- Status
- Context
- Decision
- Alternatives
- Consequences
- Decision rationale
- Evidence/review triggers

Explain what future measurements or observed requirements would justify reconsidering the decision.
Include specific triggers, not statements such as "when the system gets bigger."
Do not fabricate existing performance results.
==========
12. AUTOMATED VERIFICATION
==========
Create and execute an automated verification suite.
Choose appropriate tools based on the actual selected stack.
Verification should include, where relevant:
- Compilation/build
- Unit tests
- Domain invariant tests
- API contract tests
- Transaction integration tests
- Database migration tests

- Application rollback compatibility test
- Concurrent command tests
- Formatting/lint/static analysis
- Mermaid syntax validation
- PDF quality validation
Implement a reproducible local load-test harness.
Use exactly 764 concurrent virtual users/tasks for the AD-065 performance experiment when local resources and test-tool capabilities permit.
Record:
- Hardware/runner environment
- Server/database setup
- Exact request mix
- Concurrency model
- Test duration
- Warm-up
- Number of samples
- p50, p95, p99 if available
- Error rate
- Throughput
- Limitations
Separate:
- Completed test
- Passed test
- Failed test
- Skipped test
- Test not executable in the current environment
If resources do not support 764 concurrent tasks, do not invent a result.
Produce a reproducible test command and clearly mark p95 compliance as NOT VERIFIED.
If testing is possible, compare the observed p95 against <= 4.5 seconds.
Fix implementation problems identified by the tests and rerun affected checks.
Keep genuine failure logs where they demonstrate a problem that was subsequently corrected.
==========
13. TECHNICAL DEBT REGISTER
==========
Create a debt register with at least three evidence-backed entries.
For each entry:
- Debt ID
- Relevant source location
- Actual evidence
- Impact: 1–5
- Urgency: 1–5
- Effort: 1–5
- Priority
- Proposed action
- Regression safety
Use a transparent prioritization method.
For example:
PriorityScore = Impact * Urgency / Effort
Acknowledge limitations of such a heuristic.
Explain why the highest-priority debt should be addressed first.
Include primitive obsession in the debt analysis when justified by the captured baseline code.
Do not describe a fully resolved refactoring as unresolved debt.
==========
14. AI-ASSISTED DESIGN REVIEW
==========
Perform an actual AI-assisted design review.
Preserve the genuine main instruction supplied by the user as part of the AI Usage Log.
You may also create shorter sub-prompts for:
- Architecture alternatives

Phụ lục C · Nguyên văn chỉ dẫn được cung cấp

- Boundary analysis
- Refactoring review
- Rollback design
- Test strategy
- Evidence validation
Save each meaningful prompt and relevant output excerpts.
Recommended directory:
assignment/AD-065/evidence/ai/
The review MUST include:
1. The actual prompt used
2. Relevant actual AI output
3. At least two AI suggestions evaluated against code or requirements
4. The decision to keep, modify, or reject each suggestion
5. The exact supporting evidence
6. A statement of remaining uncertainty
Examples of proposals to critically assess:
- "Split every domain into microservices"
- "Use an automatic destructive DOWN migration during rollback"
- "Move critical validation to the client"
- "Assume a cache automatically guarantees p95 <= 4.5s"
These are examples only. Evaluate actual suggestions made during the review.
Do not fabricate separate AI conversations, timestamps, or tools.
If the CLI cannot distinguish sub-agent outputs from its own reasoning, label these accurately as a single Codex-assisted review rather than inventing independent reviewers.
The AI Usage Log must be truthful and sufficiently detailed for academic verification.
==
15. TRACEABILITY
==
Create a traceability matrix:
Driver / Requirement
→ Architecture decision
→ Boundary / Contract
→ Exact code location
→ Planned or executed test
→ Evidence / Result
Use real repository-relative paths.
Where practical include file line numbers and commit SHA.
Use a stable inspected source commit for teammate repository context.
Use the actual new branch commit for original implementation evidence.
Clearly distinguish:
- Existing code
- Generated code
- Refactored code
- Tests executed
- Tests only planned
==
16. REPORT GENERATION
==
Generate one professional, submission-quality PDF.
Filename:
BT2_23001901_AD-065.pdf
Language:
Vietnamese, except the two completely English diagrams.
Paper:
A4 portrait.
Maximum:
7 pages of main content.
The AI Usage Log appendix does not count toward this limit.
Use a readable, academically appropriate Vietnamese Unicode font embedded in the PDF.

Phụ lục C · Nguyên văn chỉ dẫn được cung cấp

Ensure correct Vietnamese diacritics.
Avoid broken characters, clipped tables, overflowing paragraphs, and tiny illegible code.
Suggested layout:
Page 1:
- Student identity and declaration
- Scope and evidence baseline
- Three architecture drivers
- Performance quality scenario
Page 2:
- Module View diagram
- Module responsibilities
- Dependency direction
- Business-rule ownership
Page 3:
- Runtime View diagram
- Schema deployment and rollback scenario
- Data ownership and transaction boundaries
- Contract and invariant
Page 4:
- Architecture alternatives
- Complete ADR
- Trade-offs and evidence triggers
Page 5:
- Code quality analysis
- At least three evidence-backed issues
- Primitive obsession
- Before/after refactoring
Page 6:
- Regression safety and executed verification
- Debt register
- Prioritization rationale
- Performance evidence or limitations
Page 7:
- AI-assisted design review
- Two corrected/rejected AI suggestions
- Traceability matrix
- Overall conclusion
- Evidence references
Appendix, outside the main 7-page limit:
- AI Usage Log
- Actual prompts
- Selected authentic AI outputs
- Review decisions
- Verification references
You may adjust the allocation as necessary.
The seven-page main report must remain coherent and readable.
Use citations to:
- Repository files
- Source commits
- Actual requirements
- Local test artifacts
- Relevant specifications
Do not cite files or code that do not exist.
Do not invent external academic references.
Every major factual claim about the repository must be traceable.
Keep the PDF report focused on architectural reasoning rather than pasting source code.
==========
17. PDF TOOLING AND QUALITY GATES
==========

Phụ lục C · Nguyên văn chỉ dẫn được cung cấp

Use available reliable document-generation tools, for example:
- Markdown + Pandoc + LaTeX
- Typst
- HTML/CSS + PDF renderer
- Another reproducible PDF toolchain
Select based on installed capabilities.
Use Mermaid CLI or an equivalent genuine Mermaid renderer for diagrams.
Prefer SVG for sharp printed diagrams.
Do not hand-draw a diagram and claim it was rendered by Mermaid.
Automatically verify:
1. The PDF exists at the required path.
2. The exact filename is correct.
3. Main content is <= 7 pages.
4. Appendix pages are clearly marked.
5. Vietnamese Unicode is rendered correctly.
6. Diagram 1 is present.
7. Diagram 2 is present.
8. Both diagrams are readable on A4.
9. Both diagrams contain English only.
10. Student identity is correct.
11. Team/project/variant are correct.
12. Declaration text is exact.
13. Source paths and commits exist.
14. Evidence statements match actual results.
15. Tables are not cut off.
16. No placeholder content remains.
17. No invented performance results appear.
Render the final PDF to page images and inspect every page.
Where possible perform text extraction checks and layout checks.
Correct errors and regenerate.
If a quality gate cannot be satisfied, explicitly record the blocker rather than reporting a false success.
==========
18. PROJECT OUTPUT STRUCTURE
==========
Target structure:

```
assignment/AD-065/
  README.md
  implementation/
  diagrams/
    module-view.mmd
    runtime-view.mmd
    module-view.svg
    runtime-view.svg
  evidence/
    repository-baseline.md
    source-analysis.md
    tests/
    performance/
    rollback/
    ai/
  report/
    report.md
    build-report.*
  submission/
    BT2_23001901_AD-065.pdf
  verification-summary.md
```

The exact implementation substructure may vary by selected technology.
Keep temporary build files out of the submission folder.
Only the final PDF is intended for Google Classroom submission.

Phụ lục C · Nguyên văn chỉ dẫn được cung cấp

Do not submit the GitHub branch URL as a substitute for the PDF.
==========
19. AUTONOMOUS EXECUTION POLICY
==========
Execute the workflow in this order:
PHASE A — RESEARCH
- Inspect repository, branches and SRS
- Record baseline commits
- Select suitable implementation scope
PHASE B — ISOLATION
- Create my independent assignment branch
- Prepare source and evidence directories
PHASE C — IMPLEMENTATION
- Build original authoritative server vertical slice
- Establish a traceable baseline
- Implement domain ownership and transaction rules
- Implement safe schema evolution/rollback
PHASE D — REFACTORING
- Capture actual before-state evidence
- Add regression tests
- Refactor primitive obsession
- Preserve observable behavior
PHASE E — VERIFICATION
- Build and run tests
- Execute integration and rollback tests
- Run reproducible concurrency/performance tests if feasible
- Repair defects and rerun tests
PHASE F — ARCHITECTURE ANALYSIS
- Produce architecture drivers
- Produce quality scenario
- Produce boundary analysis
- Compare alternatives
- Write ADR
- Produce debt register and traceability
PHASE G — AI REVIEW
- Run focused design/refactoring prompts
- Capture authentic outputs
- Verify at least two suggestions against evidence
- Document accepted, modified and rejected proposals
PHASE H — DIAGRAMS AND PDF
- Generate two English Mermaid diagrams
- Render and inspect them
- Write the Vietnamese report
- Generate the final PDF
- Enforce the seven-page main-content limit
- Inspect all PDF pages
PHASE I — FINAL AUDIT
- Check every rubric requirement
- Verify source citations and code locations
- Verify file existence
- Verify test results
- Commit changes to my branch
- Push my branch if permitted
Iterate through implementation, testing, and repair as needed.
Do not ask for routine confirmations.
Never overwrite unrelated work.
Do not perform destructive remote Git operations.
Never bypass failing tests merely to make the result look successful.

Phụ lục C · Nguyên văn chỉ dẫn được cung cấp

If an external tool or dependency is unavailable, first try an equivalent local alternative. Document any genuinely unresolved blocker.

==========
20. FINAL ACCEPTANCE CHECKLIST
==========

The final result must cover ALL rubric criteria.

[ ] Exactly 3 architecture drivers
[ ] Full 6-part Performance scenario
[ ] 764 concurrent users/tasks
[ ] p95 target <= 4.5 seconds
[ ] Operational retention = 7 days
[ ] Change-request response = 12 hours
[ ] Performance trade-off
[ ] Mermaid Module View with >=5 components
[ ] Mermaid Runtime View with failure path
[ ] English-only diagram text
[ ] Responsibilities and dependency directions
[ ] Business-rule ownership
[ ] Explicit data ownership
[ ] Transaction boundary
[ ] API/event contract
[ ] Server/database invariant
[ ] At least 2 architectural alternatives
[ ] Complete ADR
[ ] Evidence-based ADR review triggers
[ ] Real code paths and exact commit evidence
[ ] At least 3 verified code-quality concerns
[ ] Primitive obsession analysis
[ ] Before/after refactoring
[ ] Observable-behavior preservation
[ ] Regression tests
[ ] Debt register with >=3 entries
[ ] Impact/Urgency/Effort scores
[ ] Explained priority order
[ ] Genuine AI-generated review material
[ ] At least 2 AI proposals modified or rejected
[ ] Evidence for review decisions
[ ] Complete traceability
[ ] Accurate AI Usage Log
[ ] Original code on my branch
[ ] No teammate source copied
[ ] Build/test results honestly reported
[ ] Safe schema rollback demonstrated or limitation stated
[ ] Vietnamese PDF
[ ] Correct student information
[ ] Exact declaration
[ ] <=7 pages of main content
[ ] AI Usage Log in separately marked appendix
[ ] Readable A4 diagrams
[ ] Exact required PDF filename

==========
21. FINAL RESPONSE FORMAT
==========

When complete, provide:

1. Branch name and final commit SHA
2. Final PDF absolute local path
3. Whether Git push succeeded
4. Implementation summary
5. Tests passed/failed/skipped

Phụ lục C · Nguyên văn chỉ dẫn được cung cấp

6. Actual load-test p95, or NOT VERIFIED
7. Rollback verification result
8. PDF total pages and main-content pages
9. Location of AI Usage Log
10. Remaining blockers or uncertainties
If any acceptance criterion fails, clearly report it.
Do not call the PDF "ready to submit" unless all document-level requirements are satisfied and the evidence is authentic.
Start executing PHASE A immediately.